

L22 — Chunking Strategies · guida agli script

Serata infrasettimanale (🎤 + 📄 + 📊, 2h): arrivano i **documenti lunghi** di Lumen e si impara a spezzarli. Col docente si scrivono `chunk_fixed` e `chunk_recursive` in `working/chunk.py` (si guarda e si replica); `chunk_semantic` è **già nel file** e il docente la mostra in demo (si usa, non si scrive). Poi gli esperimenti: si girano le quattro manopole in testa al file (`STRATEGIA` , `CHUNK_SIZE` , `OVERLAP` , `SOGLIA`), a ogni giro lo script ri-chunka, ri-embedda, **ricrea la collection** `lumen_chunks` e rilancia le tre query note — e l'esito si annota nella tabella commentata in fondo al file. Niente notebook, solo script `.py`.

Il `check_lab22.py` è **strumento del docente**, fuori dal percorso studente (il “fatto quando” degli esperimenti è l'output osservabile): serve per la preparazione/regressione dei materiali e per la diagnosi rapida di uno studente bloccato. La verifica base è **istantanea e a zero costi** (le funzioni di chunking sono Python puro, le trappole si controllano sulle stringhe); il giro reale via API sta dietro `--live`.

Ruolo di ogni file

Legenda: 🧰 ambiente/toolkit riusabile · 🎤 codice di riferimento che il docente live-coda · 📄 da completare dagli studenti · 📄 documenti/dati di esempio.

File	Ruolo	Cosa fa
<code>requirements.txt</code>	🧰	Dipendenze (<code>chromadb</code> , <code>openai</code> , <code>numpy</code> , <code>python-dotenv</code>). Stasera il backend è <code>openai</code> per tutti: <code>sentence-transformers</code> NON serve.
<code>.env.example</code>	🧰	Template per <code>OPENAI_API_KEY</code> — la stessa chiave di L13–L21 , nessuna registrazione nuova.
<code>CONSEGNA.md</code>	🧰	Promemoria operativo della serata per gli studenti.
<code>scaffolding/aikit/embeddings.py</code>	🧰	Il modulo di L19: <code>embed()</code> e <code>cosine_similarity()</code> — stasera riusato anche DENTRO <code>chunk_semantic</code> . Non si tocca : si importa.
<code>scaffolding/aikit/search.py</code>	🧰	Il brute-force di L20, nel toolkit dalla L21. Stasera non si usa: resta nel toolkit.

File	Ruolo	Cosa fa
scaffolding/aikit/vectorstore.py		Il modulo scritto dagli studenti a L21 , promosso nel toolkit: <code>crea_collection()</code> (riparte da zero — la novità di stasera), <code>apri_collection()</code> , <code>indicizza()</code> , <code>search()</code> . Non si tocca: si importa.
scaffolding/make_dataset.py		(Ri)genera i 3 documenti lunghi in <code>dataset/</code> e verifica le trappole (posizioni dei tagli tarate al carattere, documentate nel docstring). Deterministico.
dataset/guida-smart-working.txt		La policy HR di Lumen (5.100 caratteri) — dentro: il contributo postazione a cavallo di due paragrafi (trappola B).
dataset/policy-resi-garanzia.txt		Resi, rimborsi, garanzia (5.396 caratteri) — dentro: la frase dei saldi a cavallo di un taglio <code>fixed(500,0)</code> (trappola A).
dataset/manuale-lumadesk.txt		Il manuale LD-200/LD-210 (5.681 caratteri) — dentro: la procedura E07 in un paragrafo lungo dai molti argomenti (trappola C).
working/chunk.py		L'unico file della serata. TODO 1-2 col docente (<code>chunk_fixed</code> , <code>chunk_recursive</code> — si replica); <code>chunk_semantic</code> già scritta (demo docente); il main fa tutto il giro (chunk → embed → indicizza → tre query) guidato dalle 4 manopole in testa; in fondo la tabella esperimenti da compilare. Riferimento in <code>solutions/</code> .
scaffolding/check_lab22.py		Strumento del docente: <code>[OK]/[FAIL]</code> su taglie, overlap e trappole (base: istantaneo, zero rete); <code>--live</code> fa semantic vero + tre giri di indicizzazione su un DB usa-e-getta (~\$0.002); <code>--solutions</code> verifica le soluzioni. Non compare nel percorso studente.

File	Ruolo	Cosa fa
<code>scaffolding/misura_costi.py</code>		Strumento del docente: la tabella token/costo per configurazione citata in slide, da chiamate vere (~\$0.001 a lancio).
<code>solutions/</code>		Le versioni complete dei soli file di <code>working/</code> (stesso nome, stessi import).

La cartella `chroma/` (il DB su disco, con la collection `lumen_chunks`) nasce al primo lancio e NON si versiona (`.gitignore`); si **ricrea a ogni giro** ed è la base del RAG di L23 — a fine serata non va cancellata.

Setup

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env # dentro: la stessa chiave OpenAI di L13-L21
```

Nota per chi distribuisce (docente): agli studenti la cartella arriva **senza** `.env`, `.venv` e `chroma/` (git non li versiona). Niente corpus embeddato stasera: i documenti sono in chiaro in `dataset/` e gli embedding si calcolano a ogni giro.

Come si prova

Tutto si lancia dalla cartella `scripts/`:

```
python working/chunk.py # un esperimento = un lancio (~$0.0001-0.0002 a giro)
python scaffolding/check_lab22.py # verifica working/: istantanea, zero costi
python scaffolding/check_lab22.py --live # semantic vero + 3 giri su DB usa-e-getta (~$0.002)
python scaffolding/check_lab22.py --solutions # per il docente: verifica solutions/
python scaffolding/make_dataset.py # rigenera i documenti + verifica delle trappole
python scaffolding/misura_costi.py # la tabella dei costi per configurazione (~$0.001)
```

Le idee da trasmettere (coi numeri misurati, luglio 2026)

- 1. Il documento intero è l'unità sbagliata.** Sui 3 documenti interi le tre query “trovano” il documento giusto ma con score piatti (0.466 / 0.421 / 0.380, secondi posti a 0.33–0.39) e il “risultato” è un testo di 5.000+ caratteri: il retrieval deve trovare il **pezzo**, non il documento.
- 2. fixed è cieco, e si vede.** A 500/0 tutte e tre le query tornano tagliate: il chunk dei saldi si interrompe ESATTAMENTE su “...la finestra di reso” (il taglio cade 10 caratteri prima di “14 giorni” — tarato apposta in `make_dataset.py`); il contributo esce con la scadenza senza l'importo; la procedura E07 si tronca su “per 30 se”.
- 3. L'overlap ricuce il testo, non garantisce il ranking.** Con 500/150 la frase dei saldi torna intera in testa (0.684); il chunk del contributo ricucito (importo E scadenza) ESISTE ma resta terzo — davanti c'è un frammento più “concentrato”. E si paga: +42% di token (6.853 vs 4.831).
- 4. recursive rispetta la forma, non il discorso.** 45 chunk (123–481 caratteri), mai una parola rotta: saldi 0.695 ✓ e E07 0.735 ✓ — ma il contributo resta tagliato (0.639): l'informazione è a cavallo di DUE paragrafi e recursive taglia proprio lì, per definizione.
- 5. semantic taglia dove cambia il discorso — e su questo corpus vince.** Similarità tra frasi adiacenti nel manuale: dentro una sezione 0.45–0.66 (E05→E07 0.628, E07→E09 0.660), ai confini 0.24–0.36 (manutenzione→CODICI DI ERRORE 0.236): con soglia 0.4 i tagli cadono da soli sui confini di sezione. È l'unica configurazione provata che porta a casa TUTTE e tre le query (0.699 / 0.527 / 0.665) — il contributo si salva perché le due frasi si somigliano e il chunk scavalca il confine di paragrafo. Con soglia 0.5 il contributo si rompe di nuovo: la soglia è una manopola vera.
- 6. Spezzare è gratis, la ridondanza si paga.** Indicizzare i 3 documenti: interi 4.804 token (\$0.000096), fixed 500/0 4.831 (\$0.000097), recursive 4.765 — stessi caratteri, stesso conto. L'overlap costa (+42% a 500/150, +49% a 300/100); semantic quasi raddoppia (9.389 token, \$0.000188: si embeddano anche le frasi per decidere i tagli).
- 7. Il metodo è la lezione.** Cambia UNA manopola → rilancia → leggi il 1° risultato per intero → annota ✓/✗/✓. La config giusta esiste per QUESTO corpus e QUESTE domande; su un corpus diverso la tabella si rifà da capo.

Percorso in aula (le slide sono la regia: ogni switch ha la sua slide)

- 1. Teoria (📊, 40')**: la prova sui documenti interi (i numeri del punto 1), le tre strategie, il disegno della stessa pagina spezzata nei tre modi, trade-off e costi misurati.
- 2. Live coding (🔧, 45')**: setup + `chunk_fixed` (TODO 1, si replica) → `chunk_recursive` (TODO 2, si replica) → demo `chunk_semantic` (si guarda); dopo ogni funzione, lancio su un documento vero e lettura dei tagli stampati.
- 3. Esperimenti (🖱️, 30')**: le 4 manopole, un giro = una riga della tabella in fondo a `working/chunk.py`; fatto quando almeno una configurazione porta in testa la risposta integra per ognuna delle tre query — e si sa dire quale manopola ha fatto la differenza.
- 4. Wrap-up (📊, 5')**: il reveal della tabella misurata, “si sceglie sperimentando”, ponte a L23 (i chunk di `lumen_chunks` dentro un prompt: il primo RAG completo del corso).